CARELINK FOLLOW-UP · DANIEL NG`ANDU · MANAGER SI, SOFTWARE ENGINEERING

Bring back patients who missed appointments, at 1,500 clinics that are often offline

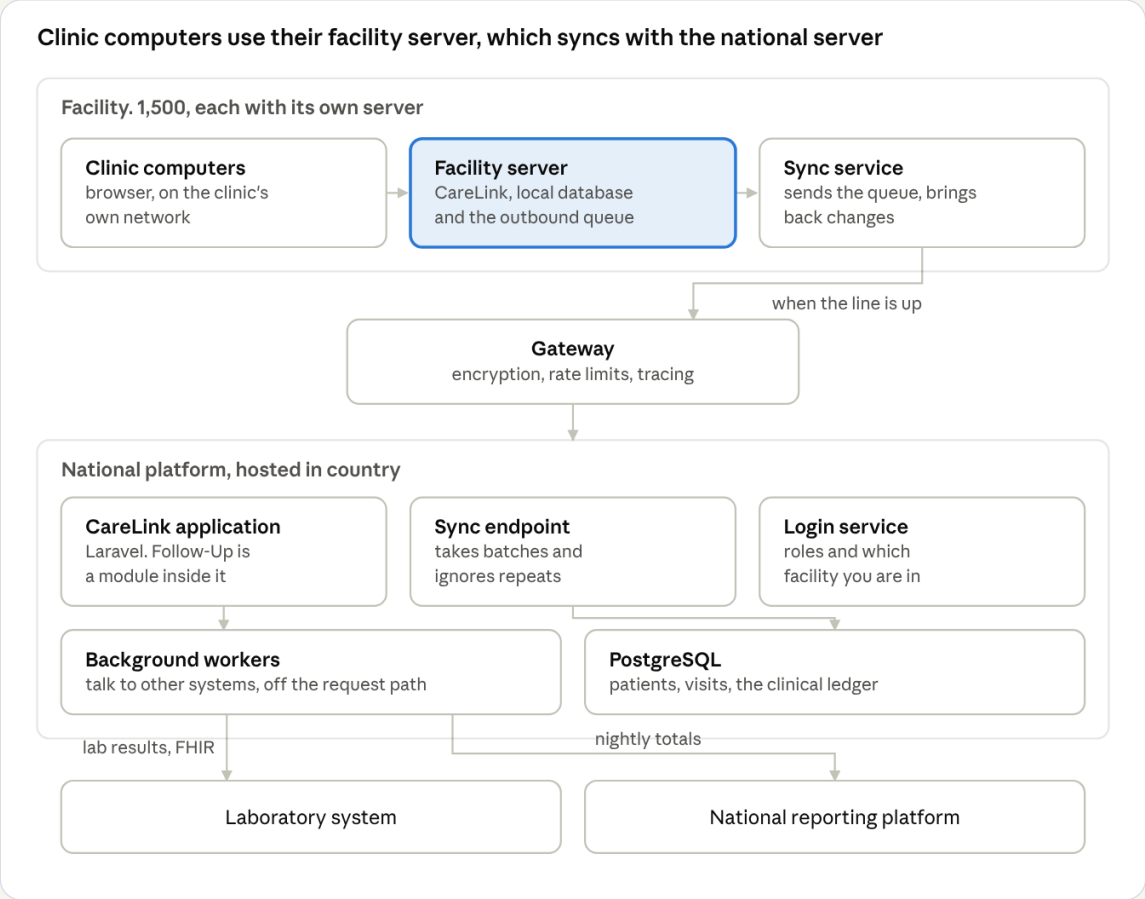
The problem as I read it

- 1,100 of 1,500 facilities have unreliable internet; 300 are offline for days
- Old computers, 4 GB of RAM, staff using the keyboard between patients
- Lists over 10,000 patients at the busiest sites
- Releases go out while clinics are open

Where the brief was unclear, I assumed

- Every facility runs its own CareLink server, as today; Follow-Up is a module on it
- A patient belongs to one facility; transfers are recorded
- Nine months means the first facilities, then phased rollout
- “More than 7 days” is strict. The pack’s sample shows 8 days as “missed”; I followed the written rule

Each clinic works on its own facility server, which syncs with the national server



One visit, clinic to national and back

1. The clinician records a visit. It is saved on the facility server with its own reference number.
2. When the internet is up, the facility server sends its queue to the national server. A repeat of the same reference is ignored.
3. The national server checks who sent it, adds it to the clinical ledger and updates the follow-up list.
4. The next sync brings changes from elsewhere back to the facility, including the current user list.

Lab results arrive through an adapter using HL7 FHIR. Totals go to the reporting platform nightly. All data stays in country.

Three decisions I would defend

Add to a ledger, never overwrite

Rejected: last change wins

When two edits clash, last change wins quietly destroys one of them. For a clinical record nobody finds out until it matters. A ledger costs more storage; it never loses a visit.

Build on the facility servers

Rejected: an offline browser app

Every facility already runs a CareLink server, so the hardware cost is paid. Clinic computers keep working whatever the internet does, and unsent work sits on a server with a nightly backup.

One application

Rejected: many small services

Nine people, nine months, 3,000 users at peak. Separate services buy scaling we do not need and an operating load the team cannot carry. Follow-Up is a module with clear boundaries.

THE DECISION I AM LEAST SURE ABOUT

Can we keep 1,500 facility servers in step?

Every database change must reach every facility server, and the national server must accept work from more than one version at once. The executive briefing in Section G is this exact problem.

What would change my mind

How many facility servers run old versions today, how many update remotely, and how often an update fails half way. I would measure it in the pilot.

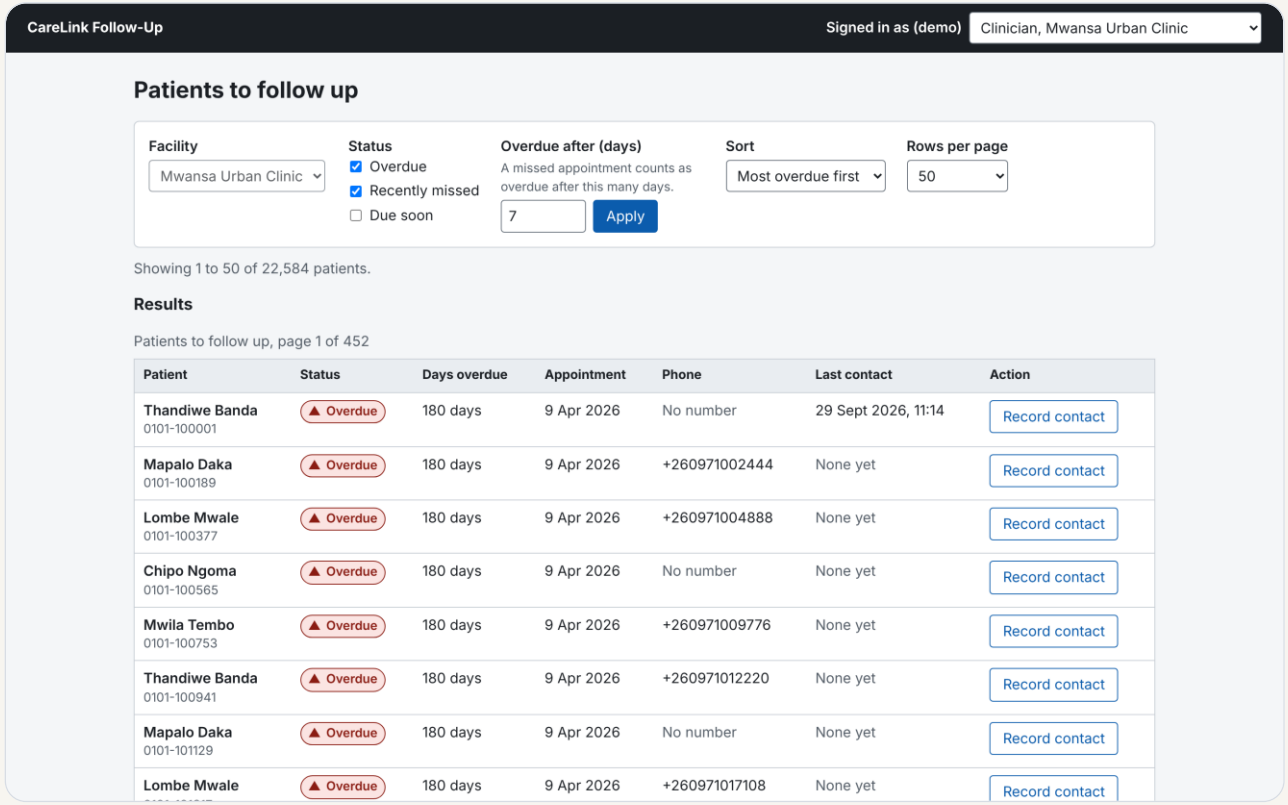
If most update cleanly

Waves of a few hundred servers a week, each checked before the next. Two-step database changes keep old and new working together.

If many lag or fail

First build an updater that checks itself and rolls back on its own, before any new feature needs a database change.

What I built: the follow-up list, end to end



27 s → 0.2 s

to show the list on an old, slow computer, once it loads 50 rows a page instead of 10,000

2.9 MB → 15 KB

sent over the clinic’s connection for one screen

About 40 ms

for a page from 100,000 patients and 400,000 visits, filtered in the database

Laravel and React. Nothing stubbed. Added a retry-safe “record contact” call. Rule checked against an independent count; screen tests pass; Laravel tests.

What my code review would block before merge

Back end: patient service

- **Search box can read or delete any record** (queries built from typed text)
- **Database password in the code,** on the all-powerful account
- **Saving a visit hides every error,** so lost visits go unnoticed

Also blocked: national ID written to logs; connections never closed

Front end: follow-up list

- **Refetches the whole list every 3 seconds,** about 1,000 requests a second at peak
- **Blank screen on first load,** no error and no retry
- **“Mark contacted” fails silently,** and keyboard users cannot reach it

Also: status shown by red text only; no field labels

I refactored the back end. The screen’s failures are visible, so staff report them. The back end’s are silent and permanent: a leaked database or a lost visit cannot be undone.

OFFLINE WORKING AND DUPLICATES, IN PLAIN WORDS

Every visit gets a receipt number

Given when it is saved. Send it twice and it counts once, so a dropped connection cannot double a record.

Nothing clinical is deleted

Duplicates already there are marked, not erased. Duplicate patients are merged by a person, and a wrong merge can be undone.

When edits clash, a person decides

The screen shows both values, who changed each and when. The software never picks a winner quietly.

No clinic is locked out

The facility server checks logins itself, so staff sign in as usual when the internet is down. Patient data stays on that server, encrypted, not on clinic computers.

With another week

Build the facility to national sync

Then replay the 16:00 scenario twice in a test and prove nothing doubles.

Measure how facility servers take updates

How many lag or fail settles the question on slide 4.

Run every test on every change

An automated pipeline for API and screen, the first step of my 90-day plan.

Load test at 3,000 users, on old hardware

And try the screen with two clinics’ staff before anyone else does.